Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries and the course modules. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Introduction to the cell culture fed-batch process simulator

The simulator provides in silico data to test the machine learning tools discussed during the course. It mimics the behaviour of a fed-batch cell culture process with only a few components:

- **Cells** produce the product. They consume glucose to sustain their metabolism and produce lactate as a by-product. The cell population is described by the viable cell density (VCD), typically expressed in million cells / mL.
- **Glucose (Glc)** is consumed by the cells and is fed continuously to the process (F_Glc). Too low glucose concentrations slow down cell growth and product formation. Too high glucose concentrations are toxic and accelerate cell death.
- **Lactate (Lac)** is a by-product of the cells. It is toxic, so high lactate concentrations slow down the cell metabolism and accelerate cell death.
- **Product (Titer)** is produced by the cells. The faster the cells grow, the less product they make.

The simulator integrates the following equations to create the in silico data.

Model equations

- Viable cell density: $\frac{dVCD}{dt} = (\mu_g - \mu_d)\, VCD$
- Glucose: $\frac{dGlc}{dt} = -k_{Glc}\, \frac{Glc}{Glc+0.05}\, VCD + F_{Glc}$
- Lactate: $\frac{dLac}{dt} = k_{Lac}\, VCD$
- Product (titer): $\frac{dTiter}{dt} = k_{Prod}\, \frac{Glc}{Glc+K_{g,Glc}} \left(1 - \frac{\mu_g}{\mu_{g,max}}\right)^2 VCD$

where

- growth rate: $\mu_g = \mu_{g,max}\, \frac{Glc}{Glc+K_{g,Glc}}\, \frac{K_{I,Lac}}{Lac+K_{I,Lac}}$
- death rate: $\mu_d = \mu_{d,max} \left(1 + \frac{\varphi}{1+\varphi}\right) \frac{Lac}{Lac+K_{d,Lac}}$
- glucose toxicity: $\varphi = e^{0.1\,(Glc-75)}$

The term $(1 - \mu_g/\mu_{g,max})^2$ couples production to growth: fast-growing cells produce little product, cells in the stationary phase produce most of it. Glucose is fed at the rate F_{Glc} between the feed start and the feed end day. Time is handled in hours inside the simulator and reported in days.

You can change the kinetic parameters of the simulator to change the process behaviour (please use the default values at the beginning).

Process parameters

The process is controlled by the following manipulated variables:

- Feed start (day): day at which the glucose feed is started
- Feed end (day): day at which the glucose feed is stopped
- Feed rate: mass rate (g/L/day) at which glucose is fed (continuous feed over 24 hours)
- Initial Glc concentration (g/L): glucose concentration at time t = 0
- Initial VCD (10^6 cells/mL): viable cell density at time t = 0

Generate a single experiment run

First we simulate one experiment with fixed model and process parameters. Change the process parameters below and re-run the cells to see how the process reacts.

```
""" Model parameters: Dictate the behaviour of the cell process """
MU_G_MAX = 0.05
MU_D_MAX = 0.025
K_G_GLC  = 1
K_I_LAC  = 30
K_D_LAC  = 50
K_GLC    = 0.04
K_LAC    = 0.06
K_PROD   = 1
MODEL_PARAM = [MU_G_MAX,MU_D_MAX,K_G_GLC,K_I_LAC,K_D_LAC,K_GLC,K_LAC,K_PROD]

""" Process parameters: Conditions at which process is run """
FEED_START = 3.0
FEED_END = 12.0
GLC_FEED_RATE = 12.0
GLC_0 = 10.0
VCD_0 = 0.3
PROCESS_PARAM = [FEED_START,FEED_END,GLC_FEED_RATE,GLC_0,VCD_0]
```

```
# Generate experiment run
t,x = simulator.predict_process(MODEL_PARAM,PROCESS_PARAM)
times = np.array(t)
run = pd.DataFrame(x,columns=["X:VCD", "X:Glc", "X:Lac", "X:Titer"])
run.head(15)
```

	X:VCD	X:Glc	X:Lac	X:Titer
0	0.300000	10.000000	0.000000	0.000000
1	0.877666	9.484266	0.777573	0.129431
2	2.389344	8.030174	2.971193	0.931287
3	5.451007	4.419283	8.432332	7.038657
4	9.954081	9.171630	19.380135	32.672120
5	14.236051	9.490923	36.991679	107.606442
6	16.097601	6.789103	59.179200	249.179059
7	15.297378	3.687971	82.056450	427.837006
8	12.870307	2.335591	102.445937	595.285539
9	10.245350	3.474163	119.046093	738.830470
10	8.071349	6.810268	132.179396	866.657963
11	6.303893	11.977601	142.485911	975.339901
12	4.872540	18.655717	150.495659	1063.742421
13	3.722240	14.563316	156.653283	1132.983899
14	2.811043	11.457108	161.330587	1185.592536



Generate a design of experiments

Despite its apparent simplicity and small number of components, the model shows a complex behaviour. In this section you can simulate the process in a broad range of process conditions.

A number of simulations defined by `NUM_RUNS` is generated. For this number of runs, a Latin hypercube design (LHD) is created to uniformly cover the 5-dimensional space of the manipulated variables. The first run is placed in the centre of the design space.

Manipulated variables

For each manipulated variable you can define the limits of the exploration space (first value: lower limit; second value: upper limit).

Note: the generated runs are saved locally as `generated/owu.csv` (the process evolution, one row per run and day) and `generated/owu_doe.csv` (the design, one row per run). The pre-generated datasets used as train and test sets in the following lectures are stored in the folder `datasets/`.

```
""" DOE Dataset definition (variable = [lower bound, upper bound]) """
""" Model parameters: Dictate the behaviour of the cell process """
MU_G_MAX = 0.05
MU_D_MAX = 0.025
K_G_GLC  = 1
K_I_LAC  = 30
K_D_LAC  = 50
K_GLC    = 0.04
K_LAC    = 0.06
K_PROD   = 1
""" Process parameters: Conditions at which process is run """
FEED_START = [1, 4]
FEED_END = [8, 12]
GLC_FEED_RATE = [5, 20]
GLC_0 = [10, 80.0]
VCD_0 = [0.1, 1.0]

""" Number of experiments to generate """
NUM_RUNS = 40

""" Filename and filepath for the dataset """
FILENAME = "owu.csv"
FILEPATH = "generated/"

# Collect parameters to dictionary
VAR_LIMS = {"mu_g_max":MU_G_MAX,
    "mu_d_max": MU_D_MAX,
    "K_g_Glc" : K_G_GLC,
    "K_I_Lac" : K_I_LAC,
    "K_d_Lac" : K_D_LAC,
    "k_Glc" : K_GLC,
    "k_Lac" : K_LAC,
    "k_Prod" : K_PROD,
    "feed_start" : FEED_START,
    "feed_end" : FEED_END,
    "Glc_feed_rate" : GLC_FEED_RATE,
    "Glc_0" : GLC_0,
    "VCD_0" : VCD_0}
```

```
# Generate Dataset
data = simulator.generate_data(VAR_LIMS, NUM_RUNS, FILEPATH + FILENAME)
# Import DOE
doe = pd.read_csv(FILEPATH+FILENAME.replace(".csv","_doe.csv"),index_col=None,
usecols =["feed_start","feed_end","Glc_feed_rate","Glc_0","VCD_0"])
# Import OWU
owu = pd.read_csv(FILEPATH+FILENAME,index_col=None, usecols = ["X:VCD", "X:Glc",
"X:Lac", "X:Titer","W:Feed"])
owu.index = pd.MultiIndex.from_product([list(range(NUM_RUNS)),list(range(15))],
names=["run","time"])
```

```
# DOE: design of the runs (one row per run)
doe
```

	feed_start	feed_end	Glc_feed_rate	Glc_0	VCD_0
0	2.500000	10.000000	12.500000	45.000000	0.550000
1	3.677411	8.948293	12.586042	36.643331	0.166513
2	3.169743	9.622924	18.717902	78.224991	0.477754
3	2.765078	11.140461	18.442434	72.139852	0.797682
4	3.471340	11.041054	14.738231	40.139376	0.181077
...	...	...	...	...	...
35	1.648302	11.418560	12.756589	27.901625	0.102161
36	3.597604	10.378306	15.228144	19.667331	0.547389
37	3.706642	8.240292	9.458184	77.218094	0.866829
38	2.810835	10.347746	19.645235	18.431605	0.619534
39	3.237216	9.114133	17.919700	34.142417	0.139321

40 rows × 5 columns

```
# OWU: process evolution (one row per run and day)
owu
```

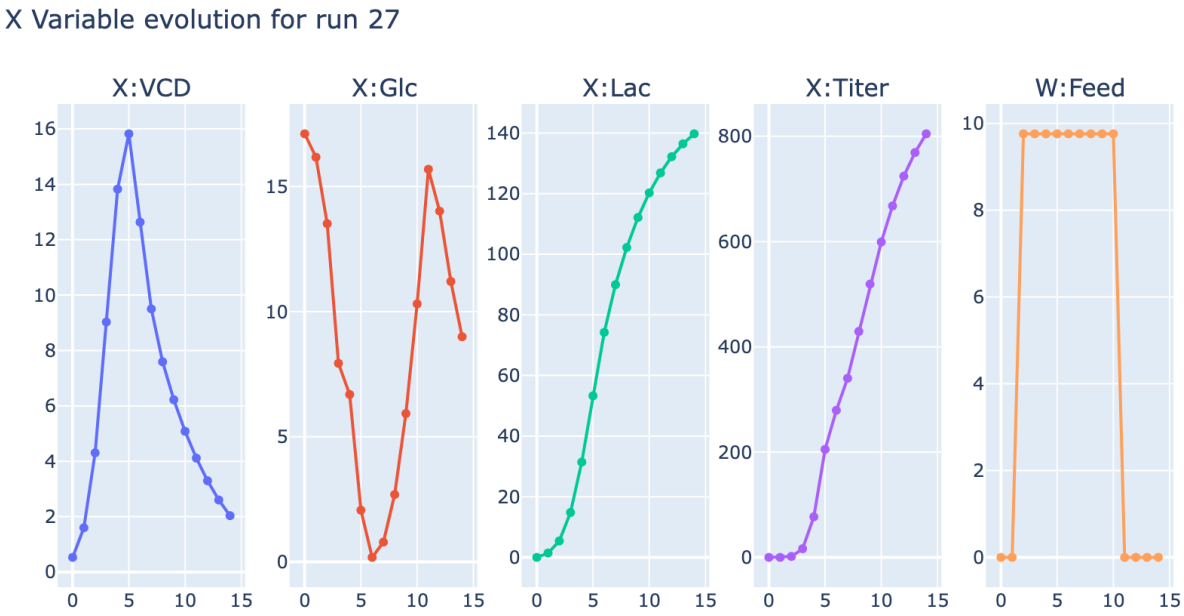
run	time	X:VCD	X:Glc	X:Lac	X:Titer	W:Feed
0	0	0.550000	45.000000	0.000000	0.000000	0.0
	1	1.725244	44.008188	1.489389	0.054632	0.0
	2	4.779558	41.089619	5.872389	1.311539	12.5
	3	10.278397	40.271478	16.487303	15.870869	12.5
	4	15.886799	40.083603	35.542541	85.824621	12.5
...	...	...	...	...	...	...
39	10	9.791774	46.245058	139.965943	1008.957267	0.0
	11	7.519302	37.985195	152.370493	1149.277817	0.0
	12	5.702837	31.681390	161.839811	1258.583370	0.0
	13	4.277633	26.927715	168.982517	1342.049360	0.0
	14	3.180691	23.378490	174.316955	1404.833665	0.0

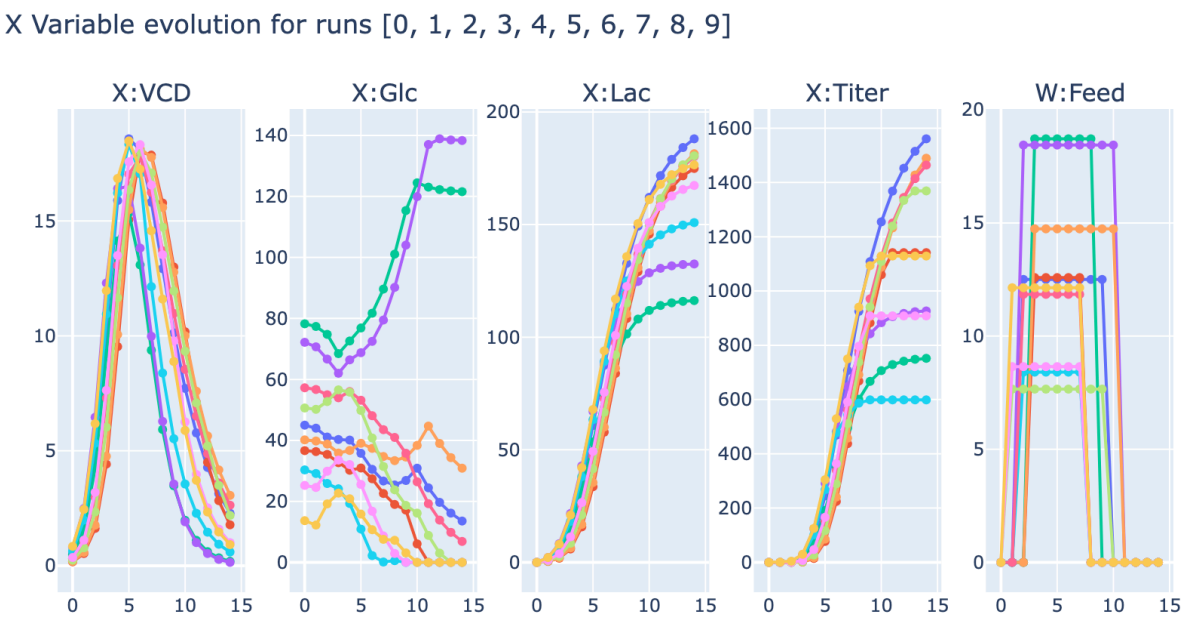
600 rows × 5 columns

Analyse single experiments

Here you can visualize particular runs from the generated dataset. Change `PLOT_RUN_ID` to decide which run is plotted and `PLOT_RUNS_IDS` to plot a group of runs on top of each other.

```
""" Select specific run to be plotted """
PLOT_RUN_ID = 27
""" Select group of run to be plotted"""
PLOT_RUNS_IDS = [0,1,2,3,4,5,6,7,8,9]
```





Analyse all experiments

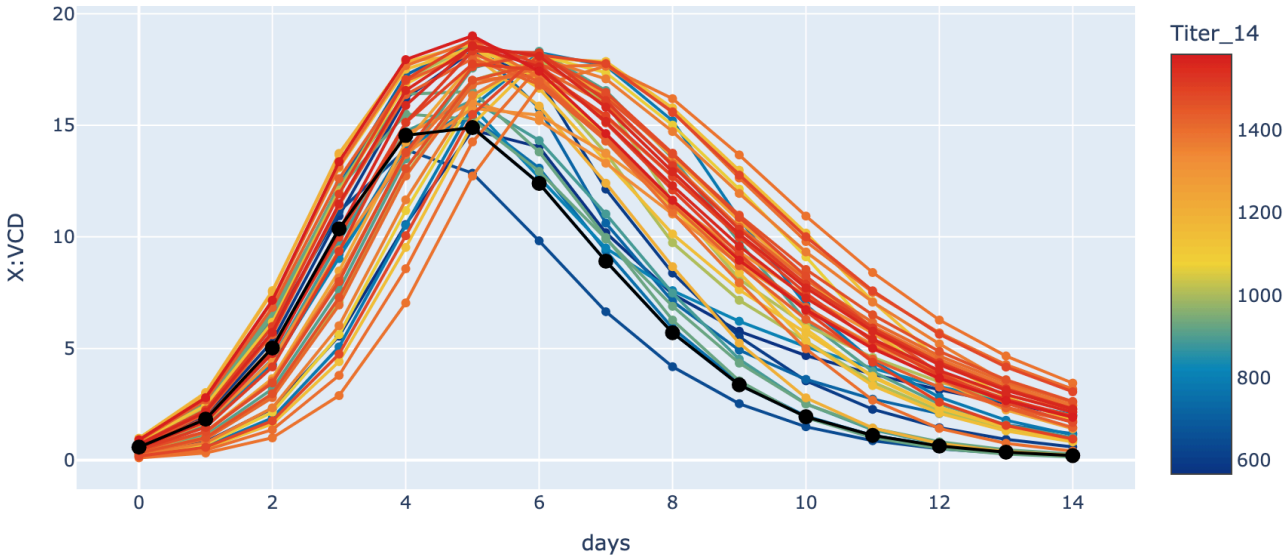
Here you can visualize all runs of the generated dataset. With `PLOT_COLOR` you decide how the experiments are coloured. The options are:

- `Run_id` runs are coloured by the order in which they appear in the dataset.
- `Titer_14` runs are coloured by the titer at day 14 of the experiment.
- `Glc_0` runs are coloured by the designed initial glucose level.
- `VCD_0` runs are coloured by the designed initial VCD level.
- `feed_start` runs are coloured by the designed feed start day.
- `feed_end` runs are coloured by the designed feed end day.
- `Glc_feed_rate` runs are coloured by the designed glucose feed rate.

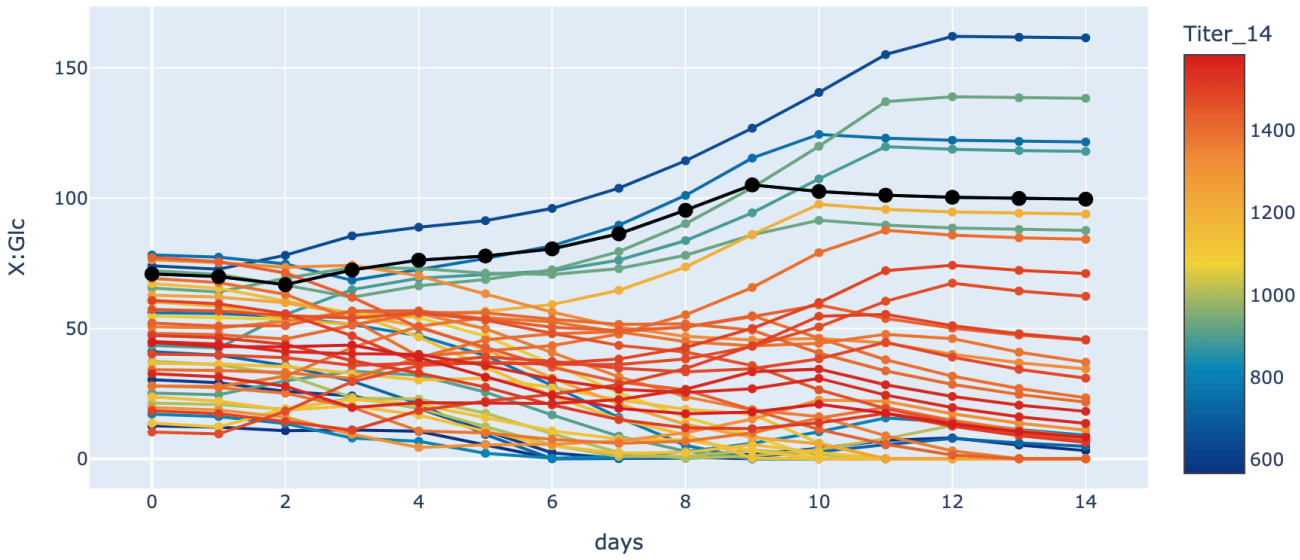
With `PLOT_HIGHLIGHT` a single run is drawn in black on top of the others.

```
""" Plot coloring option, one of (Run_id, Titer_14, Glc_0, VCD_0, feed_start,
feed_end, Glc_feed_rate)"""
PLOT_COLOR = "Titer_14"
""" Select run to be highlighted in all plots """
PLOT_HIGHLIGHT = 10
```

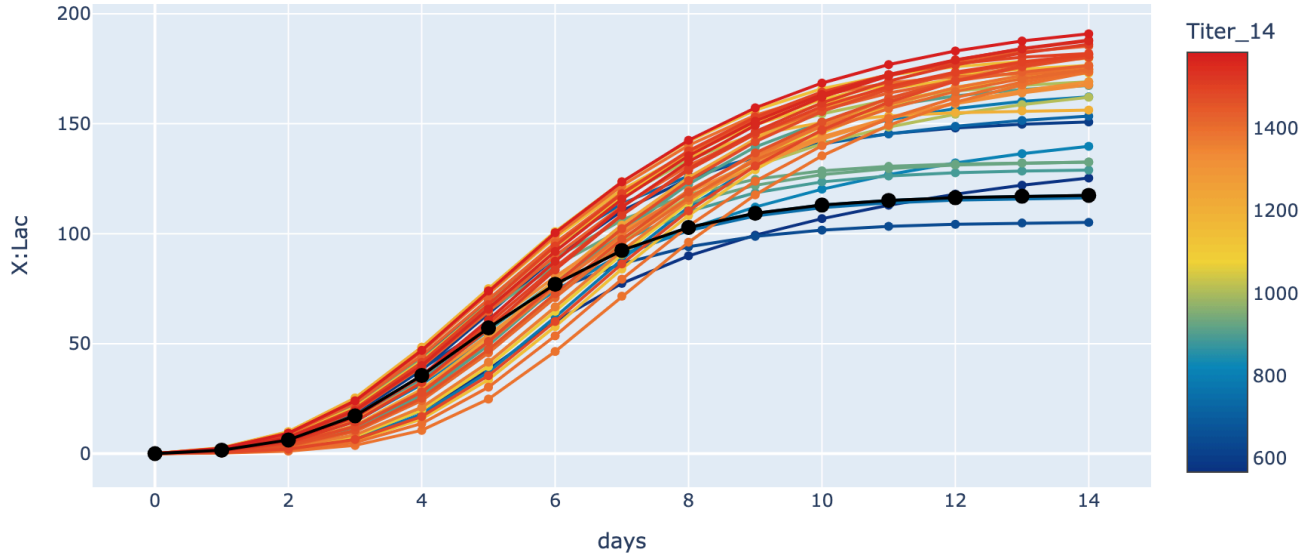
X:VCD


20
15
10
5
0
X:VCD
0
2
4
6
8
10
12
14
days
Titer_14
1400
1200
1000
800
600

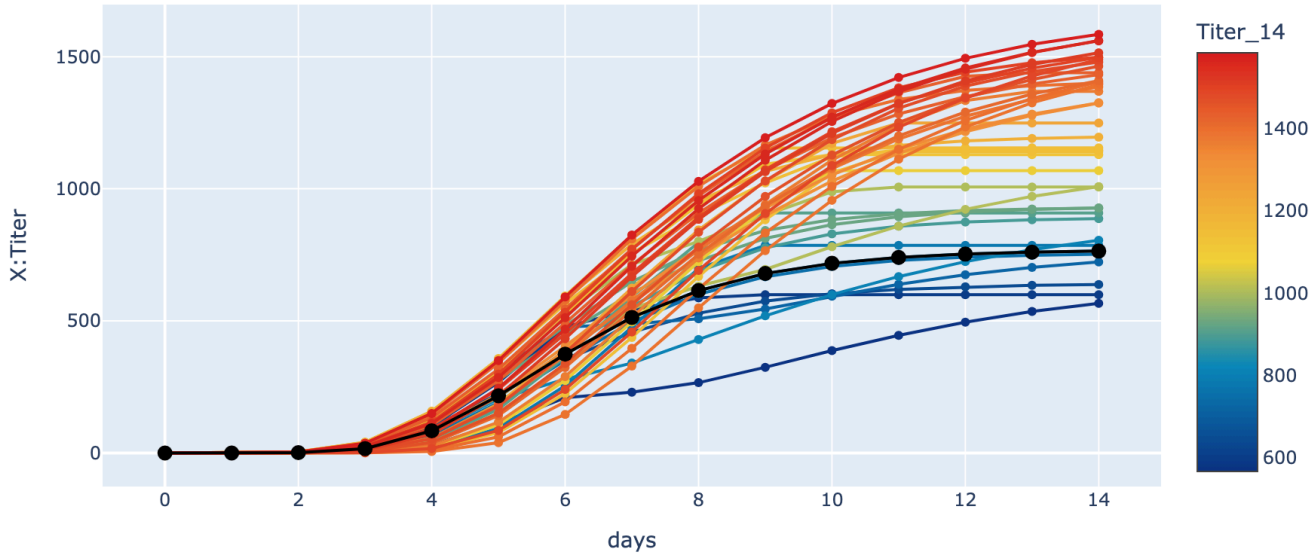
X:Glc

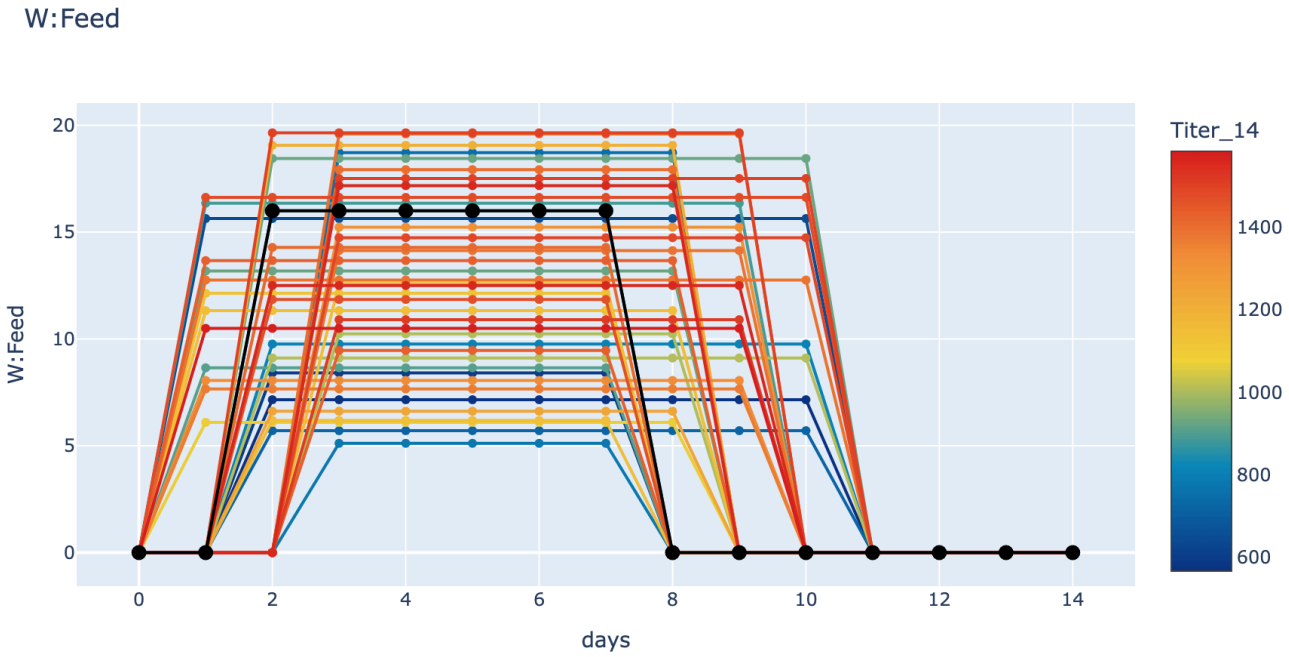

150
100
50
0
X:Glc
0
2
4
6
8
10
12
14
days
Titer_14
1400
1200
1000
800
600

X:Lac


X:Lac
200
150
100
50
0
0
2
4
6
8
10
12
14
days
Titer_14
1400
1200
1000
800
600

X:Titer


X:Titer
1500
1000
500
0
0
2
4
6
8
10
12
14
days
Titer_14
1400
1200
1000
800
600



Tasks

1. Identify which process parameters produce the highest titer in the simulated experiments (use the colouring options above).
2. Replicate the best run in the single experiment generation at the top of the notebook and try to modify the process parameters to reach an even higher titer.
3. What happens with the titer when the glucose feed rate is very high (for example 40 g/L/day)? Explain the behaviour with the model equations.

```
# Your code here
```